

SOLUTIONS FRONT-END EMULATION – CAMBRIDGE

Arm Front-End Emulation 26-Day Preparation Plan

A focused ramp-up for AMBA, SystemVerilog, waveform debug, and emulation thinking.

Role	Solutions, Front-End Emulation – Cambridge
Mentor	Anurag Sharma
Prep window	3 June–28 June 2026
Start date	29 June 2026
Primary spine	AMBA protocols: VALID/READY, APB, AXI, AHB, ACE, CHI

The target is not to know Arm's internal flow.
The target is to arrive fluent in the public fundamentals and ready to learn fast.

1. The Operating Frame

The honest target. Being “90% prepared” does not mean knowing the team’s private tools, regressions, scripts, or emulation platforms. It means being strong on the public fundamentals: AMBA behaviour, clean synthesisable SystemVerilog, waveform analysis, assertion thinking, emulation vocabulary, and professional engineering notes.

Seven Preparation Dimensions

1. AMBA and interconnect protocols
2. SystemVerilog and Verilog RTL discipline
3. Verification structure: drivers, monitors, checkers
4. Waveform analysis and deadlock diagnosis
5. Emulation mindset: synthesisable transactors
6. Computer architecture and coherency context
7. Professional intern execution and notes

Daily Non-Negotiable Loop

```
Learn -> Code -> Simulate -> Break
      -> Debug waveform -> Add assertion
      -> Write note -> Commit
```

Minimum effective day: 4 hours.

Aggressive day: 6–7 hours.

Rule: if a day has no waveform, it was not a serious prep day.

Daily Blocks

Block	Time	Purpose
AMBA / architecture concept	45–60 min	Learn the protocol or system idea.
SystemVerilog build	90–150 min	Implement the idea in clean RTL.
Verification / debug	60–90 min	Test, break, inspect waveform, and fix.
Book integration	30–45 min	Use Hennessy & Patterson or Pong P. Chu selectively.
Recall + engineering note	20–30 min	Lock the lesson into long-term memory.

Required Toolchain

VS Code, Verilator, Icarus Verilog, GTKWave or Surfer, Makefiles, Git/GitHub, Verible, and Python 3. Optional: cocotb, EDA Playground, Vivado or Quartus.

Repository Shape

```
arm_frontend_emulation_prep/
├── 00_valid_ready/
├── 01_apb_slave/
├── 02_axi_lite_slave/
├── 03_axi_lite_to_apb_bridge/
├── 04_axi4_burst_memory/
├── 05_axi_checker_sva/
├── 06_ahb_lite_slave/
├── 07_cache_coherency_model/
├── 08_chi_message_model/
├── 09_emulation_transactor_model/
├── common/
└── notes/
```

2. Phase Roadmap

Phase 1 Days 1–4

Handshake, RTL discipline, waveform fundamentals
VALID/READY, skid buffers, interfaces, SVA basics

Phase 2 Days 5–7

APB and register blocks
Setup/access timing, address decode, live peripheral

Phase 3 Days 8–13

AXI4-Lite and structured verification
Five channels, write/read paths, driver/monitor/scoreboard, bridge

Phase 4 Days 14–18

Full AXI4
Bursts, WLAST/RLAST, IDs, ordering, protocol checker

Phase 5 Days 19–21

AHB and SoC integration
AHB-Lite, protocol comparison, reset and CDC awareness

Phase 6 Days 22–24

Coherency, ACE, CHI
MESI, snoops, packet/flit vocabulary

Phase 7 Days 25–26

Emulation mindset and mock execution
Host/emulator split, file audit, professional debug cycle

Protocol Priority When Time Is Tight

#	Topic	Reason
1	VALID/READY	It is the atom of AXI channel behaviour.
2	AXI-Lite	Highest-value protocol subset for disciplined debug.
3	APB	Fastest full protocol to understand and implement.
4	AXI4 bursts	Introduces throughput, WLAST/RLAST, and counters.
5	SVA/checkers	Converts protocol rules into executable contracts.
6	Waveform drills	Builds the real debug reflex.
7	AXI-to-APB bridge	Forces protocol translation thinking.
8	AHB-Lite	Adds pipelined address/data context.
9–11	Coherency, ACE, CHI	Vocabulary and mental model for coherent systems.

Do not confuse vocabulary with competence. CHI depth without AXI discipline is fake depth. The visible evidence by 28 June is a clean repo, passing tests, annotated waveforms, and precise notes.

3. Phase 1 – Days 1–4

Handshake, RTL Discipline, Waveform Fundamentals

Day 1 VALID/READY is the atom

Focus: A transfer happens only when `VALID` and `READY` are both high on the rising clock edge. Payload must remain stable during stalls.

Build: `source.sv`, `sink.sv`, `valid_ready_if.sv`, and `tb_valid_ready.sv`. Source sends incrementing 32-bit data; sink stalls randomly.

Verify: Inspect `valid`, `ready`, `data`, and `transfer_count`. Prove data stability during every stall.

Output: Handshake note: what breaks if `VALID` drops early, and what the waveform proved.

Day 2 Backpressure and skid buffer

Focus: Downstream stalls propagate upstream. A one-entry skid buffer is the basic backpressure-tolerant element.

Build: Build `skid_buffer.sv`: empty, full, bypass, simultaneous input/output, and full throughput.

Verify: Create no-stall, one-cycle-stall, and long-stall traces. Mark input handshake, buffer capture, and output handshake.

Output: Short note on datapath/FSM separation and why the buffer preserves throughput.

Day 3 Interfaces, modports, packages

Focus: Real buses are signal bundles. Signal organisation is part of RTL quality, not decoration.

Build: Refactor the source, sink, and skid buffer to use an interface with master/slave modports. Add a passive monitor.

Verify: Monitor prints time, `valid`, `ready`, `data`, and transfer status. Match the log to the waveform.

Output: A compact SystemVerilog style note: interface purpose, modport direction, and reusable package content.

Day 4 Assertions and debug discipline

Focus: Protocols are contracts. Assertions make those contracts executable and visible during simulation/emulation.

Build: Create `vr_assertions.sv`: `VALID` hold, data stability, no-X after reset, and transfer-count rules.

Verify: Break three things intentionally: data changes during stall, `VALID` drops early, X after reset. Confirm each assertion fires.

Output: Waveform debug notes for all three failures plus a clean final commit.

Core SVA Pattern

```
property p_data_stable_when_stalled;
  @(posedge clk) disable iff (!rst_n)
    valid && !ready | => valid && $stable(data);
endproperty

assert property (p_data_stable_when_stalled);
```

4. Phase 2 – Days 5–7

APB, Register Blocks, Peripheral Thinking

APB is simple, common, and worth fully owning. The mental model is setup, access, completion. A correct APB register block becomes a reusable building block for later AXI-to-APB integration.

Day 5

APB transfer timing

Focus: Learn PADDR, PSEL, PENABLE, PWRITE, PWDATA, PRDATA, PREADY, and PSLVERR.

Build: Create `apb_if.sv`, `apb_slave_empty.sv`, and `tb_apb_basic.sv`. Use manual stimulus first.

Verify: Capture one write and one read. Identify setup cycle, access cycle, and completion cycle on the waveform.

Output: APB timing note with one hand-drawn or screenshot-based waveform annotation.

Day 7

APB checker and live peripheral

Focus: Understand error response and stable-signal requirements during ACCESS.

Build: Add APB assertions. Connect `CTRL[0]` to a counter enable, `STATUS[0]` to running, and `DATA_OUT` to count.

Verify: Check stable PADDR, stable PWDATA, legal PENABLE, and PSLVERR on invalid access.

Output: A real APB peripheral with checker, tests, waveform, and commit.

Day 6

APB register block

Focus: Memory-mapped registers are everywhere in SoC design. Learn address decode and access policy.

Build: Build four registers: `CTRL`, `STATUS`, `DATA_IN`, `DATA_OUT`. Add `apb_write` and `apb_read` tasks.

Verify: Write/read `CTRL`, write `DATA_IN` and observe `DATA_OUT`, then hit an illegal address.

Output: Register map note explaining read/write, read-only, and invalid-address behaviour.

APB Transfer Skeleton

```
SETUP  : PSEL=1, PENABLE=0
ACCESS : PSEL=1, PENABLE=1
DONE   : PSEL && PENABLE && PREADY
```

APB Register Map

Address	Register	Access	Purpose
0x00	CTRL	R/W	Enables live peripheral behaviour.
0x04	STATUS	Read-only	Reports counter running status.
0x08	DATA_IN	R/W	Input data/control payload.
0x0C	DATA_OUT	Read-only	Counter current value or derived output.

5. Phase 3 – Days 8–13

AXI4-Lite and Structured Verification

AXI-Lite is the centre of the plan. The key idea is channel independence: address, data, and response timing must not be assumed to occur in the same cycle.

Day 8 AXI five channels

Focus: Learn AW, W, B, AR, and R. AXI-Lite has no bursts and no IDs, but still has independent channels.

Build: Create `axi_lite_if.sv`, `axi_lite_pkg.sv`, and a manual `tb_axi_lite_waveform.sv`.

Verify: Pin-wiggle the interface and observe all five channels. No driver yet.

Output: A channel map note: direction, payload, handshake, and response meaning.

Day 9 AXI-Lite write path

Focus: AW and W are independent. A compliant slave handles AW first, W first, and same-cycle arrival.

Build: Implement write capture in `axi_lite_slave.sv`. Issue B only after both address and data are captured.

Verify: Produce three labeled traces: AW-before-W, W-before-AW, and same-cycle.

Output: A write-path note explaining why same-cycle assumptions are unsafe.

Day 10 AXI-Lite read path

Focus: Read flow is AR handshake, slave raises RVALID, master accepts with RREADY.

Build: Add read channel support to the register block, including invalid-address response.

Verify: Read CTRL, STATUS, invalid address, and stall RREADY for several cycles.

Output: Assertion note for RDATA/RRESP stability while RVALID is stalled.

Day 11 Driver, monitor, scoreboard

Focus: Move beyond monolithic testbenches. Separate stimulus, observation, and checking.

Build: Create `axi_lite_driver.sv`, `axi_lite_monitor.sv`, and `axi_lite_scoreboard.sv`. Use a transaction struct.

Verify: Match scoreboard logs to waveform events. Every mismatch must be waveform-traceable.

Output: A verification architecture note: driver versus monitor versus scoreboard.

Day 12 AXI-Lite assertions

Focus: Turn channel rules into SVA: stability, hold-until-ready, response stability, and no-X after reset.

Build: Build `axi_lite_checker.sv` for all five channels. Inject one deliberate bug and confirm assertion capture.

Verify: Confirm the assertion fires before relying on manual waveform inspection.

Output: One clean checker commit plus a short bug-injection log.

Day 13 AXI-Lite to APB bridge

Focus: Bridge work forces protocol translation and system-level timing thought.

Build: Build `axil_apb_bridge.sv`: single outstanding, aligned 32-bit access, read/write, basic SLVERR propagation.

Verify: Trace one AXI write becoming an APB write, then one read path. Label both sides.

Output: Deliver APB peripheral, AXI-Lite block, driver/monitor/scoreboard, checker, bridge, and traces.

AXI-Lite Channel Map

Channel	Abbrev.	Direction	Purpose
Write address	AW	Master to slave	Write address and control.
Write data	W	Master to slave	Write data and strobes.
Write response	B	Slave to master	Write acknowledgement and response.
Read address	AR	Master to slave	Read address and control.
Read data/response	R	Slave to master	Read data and response.

6. Phase 4 – Days 14–18

Full AXI4: Bursts, IDs, Ordering, Real Debug

Day 14

AXI bursts

Focus: Learn `AxLEN`, `AxSIZE`, and `AxBURST`. Bursts reduce address-phase overhead and improve throughput.

Build: Build `axi_addr_gen.sv`: start address, len, size, burst type, beat address, beat index, last.

Verify: Support INCR first, FIXED second, WRAP only as a stretch goal.

Output: Short architecture note: bandwidth versus latency and why bursts exist.

Day 15

AXI write burst memory

Focus: One AW transaction, multiple W beats, WLAST on final beat, one B response.

Build: Create `axi4_mem_write_slave.sv`: INCR bursts,WSTRB byte enables, single outstanding transaction, WLAST checking.

Verify: Run 1-beat, 4-beat, and stalled write bursts.

Output: Assertions for WLAST-on-final, no extra beat after WLAST, and BVALID only after complete burst.

Day 16

AXI read burst memory

Focus: AR handshake launches multiple R beats; RLAST marks the final read data beat.

Build: Add read burst support to the memory model.

Verify: Trace a 4-beat read burst. Randomly stall RREADY mid-burst and check RDATA/RLAST stability.

Output: Read-burst waveform note with final-beat evidence.

Day 17

IDs and ordering

Focus: Same ID responses must complete in order. Different IDs give more freedom.

Build: Build a transaction logger in the monitor for AW, W, B, AR, and R events with IDs and beat numbers.

Verify: Use IDs to visually annotate or colour-tag transactions in notes.

Output: Ordering note: what same-ID ordering means and what out-of-order does not mean.

Day 18

AXI protocol checker day

Focus: Consolidate AXI4 rules into a reusable checker and validate it against broken tests.

Build: Create `axi4_checker.sv`: VALID hold, payload stability, WLAST/RLAST position, BVALID after complete burst, no-X.

Verify: Run five broken tests: VALID drop, payload change, early WLAST, missing RLAST, early BVALID.

Output: All five bugs caught by assertions, with failing test names and root causes documented.

Burst Signals to Own

Signal	Meaning
<code>AxLEN</code>	Number of beats minus one. <code>AxLEN=3</code> means four beats.
<code>AxSIZE</code>	Bytes per beat encoded as $\log_2(\text{bytes})$.
<code>AxBURST</code>	00=FIXED, 01=INCR, 10=WRAP.
<code>WLAST/RLAST</code>	Final beat indicators for write and read burst data channels.
<code>WSTRB</code>	Byte enables for write data.

7. Phase 5 – Days 19–21

AHB, Bus Comparison, SoC Integration

Day 19

AHB-Lite

Focus: AHB uses a pipelined address/data scheme. Address phase of the next transfer overlaps the data phase of the current one.

Build: Build a minimal AHB-Lite slave register block with `HADDR`, `HTRANS`, `HWRITE`, `HREADY`, and `HRESP`.

Verify: Mark address phase, data phase, and wait state in a waveform.

Output: Comparison note: why AHB is simpler than AXI but less flexible.

Day 20

Protocol comparison and mini SoC

Focus: Write the APB/AHB/AXI/ACE/CHI comparison table from memory before checking it.

Build: Integrate AXI-Lite master testbench, AXI-Lite-to-APB bridge, and APB register peripheral.

Verify: Trace an AXI-Lite write all the way to an APB register effect.

Output: Mini SoC commit with block diagram, waveform, and protocol comparison table.

Day 21

Reset, clocks, CDC awareness

Focus: Front-end and emulation bugs often come from reset sequencing, X-propagation, clock enables, and CDC assumptions.

Build: Add reset stress tests: idle, APB transfer, AXI burst, and active stall.

Verify: After each reset: no stuck VALID/READY, no X after reset, FSM in IDLE.

Output: Reset debug checklist and evidence for each stress case.

Protocol Comparison Snapshot

Protocol	Structure	Complexity	Primary use
APB	Single control bus	Low	Peripheral registers.
AHB	Pipelined single bus	Medium	Embedded subsystems.
AXI	Five independent channels	High	High-performance IP.
ACE	AXI plus coherency extensions	High	Coherent cached masters.
CHI	Packet/flit-based layered protocol	Very high	Scalable coherent systems.

Reset drill rule. A passing reset test is not just “the sim ended.” It means no stuck handshakes, no post-reset X values, and a confirmed idle state after reset de-assertion.

8. Phase 6 – Days 22–24

Coherency, ACE, CHI, Computer Architecture Context

Day 22

Cache coherency

Focus: Learn cache line, private cache, shared memory, dirty/clean, write-back/write-through, invalidation, snoop, MESI, and coherency versus consistency.

Build: Build a two-master toy MESI model with one shared cache line and independent state registers.

Verify: Simulate: core0 read to E, core1 read to S/S, core0 write to M/I, core1 read with snoop and data forward.

Output: Timestamped waveform of all four MESI transitions.

Day 23

ACE orientation

Focus: ACE starts from AXI and adds hardware coherency: snoop address, snoop response, snoop data, coherent transaction fields, and domains.

Build: Do not implement ACE. Extend the toy model with a snoop invalidate request, ACK, and transition to Invalid.

Verify: Explain why AXI alone is insufficient for coherent cached masters.

Output: Engineering note: what ACE adds, what ACE-Lite is, and why snoops flow toward the master.

Day 24

CHI orientation

Focus: CHI is packet/flit-based and built for scalable coherent systems. Learn node types and message classes.

Build: Build a toy CHI message struct with channel, transaction ID, opcode, address, and data. Simulate a basic read transaction.

Verify: Trace RN sends REQ, HN receives, SN returns DAT, HN returns RSP/DAT to RN.

Output: Vocabulary note: RN-F, RN-I, HN-F, SN-F, REQ/R-SP/DAT/SNP, flit, opcode, credit.

Coherency Vocabulary

Term	Working meaning
MESI	Modified, Exclusive, Shared, Invalid cache-line states.
Snoop	Request for a cache to observe or change state because another agent needs data or ownership.
ACE-Lite	I/O-coherent master support without a fully snooped cache.
RN/HN/SN	Request, Home, and Slave nodes in CHI-style systems.
Flit	Packet fragment carrying request, response, data, or snoop information.

Toy CHI Message Type

```
typedef enum logic [1:0] {
    CH_REQ = 2'b00,
    CH_RSP = 2'b01,
    CH_DAT = 2'b10,
    CH_SNP = 2'b11
} chi_channel_e;

typedef struct packed {
    chi_channel_e ch;
    logic [7:0] txn_id;
    logic [7:0] opcode;
    logic [31:0] addr;
    logic [63:0] data;
} chi_flit_t;
```

9. Phase 7 – Days 25–26

Emulation Mindset and Mock Internship Execution

Day 25

Front-end emulation workflow

Focus: Understand the host software, interface/transactor layer, and emulator hardware split. Know the vocabulary without pretending to know Arm's internal flow.

Build: Audit every repo file as synthesizable RTL, simulation-only, assertion/checker, interface/package, or documentation.

Verify: Refactor the messiest testbench into driver, monitor, scoreboard, checker, and test.

Output: File classification table plus a short note on synthesizable versus simulation-only constructs.

Day 26

Mock Arm debug day

Focus: Run a professional debug cycle on a realistic AXI4 burst-memory bug.

Build: Run regression, identify failing test, open waveform, locate assertion, write bug hypothesis, fix, rerun, commit.

Verify: Regression must pass after the fix and must not hide the root cause.

Output: Final bug note, final repo cleanup, final deliverables review, and descriptive commit.

Host / Emulator Split

```
HOST SOFTWARE
  test control, sequence generation, logging,
  comparators, regression scripts

INTERFACE LAYER
  synthesizable transactors, protocol bridge,
  SCE-MI-style communication boundary

EMULATOR HARDWARE
  compiled DUT RTL, assertions, clocks/resets,
  waveform probes, embedded stimulus
```

Synthesizability Classification

Synthesizable / emulator-side candidates

always_ff, always_comb, FSMs, packed structs, parameters, generate blocks, interfaces with care, case, assign

Simulation/testbench only

Classes, dynamic arrays, queues, mailbox, semaphore, randomize/constraints, most fork/join forms, delays, UVM components, testbench initial blocks

Bug Note Template

```
Title:
  AXI write burst WLAST asserted one beat early
Symptom:
  Assertion p_wlast_final_beat fired during 4-beat INCR burst.
Evidence:
  AWLEN=3, expected beats 0--3; WLAST observed on beat 2.
Root cause:
  Beat counter compared against AWLEN-1 instead of AWLEN.
Fix:
  Compare beat_count == awlen_q.
Regression:
  APB smoke PASS; AXI-Lite smoke PASS; AXI4 burst PASS.
```

10. Daily Output Checklist

Every day must create evidence. Reading is useful only when it changes code, a waveform, an assertion, a note, or a question.

Completion Gate

- ☐ One protocol or architecture concept learned and noted
- ☐ One SystemVerilog module written or meaningfully improved
- ☐ One simulation run that completes without tool errors
- ☐ One waveform opened and inspected
- ☐ One assertion added, tested, or debugged
- ☐ One engineering note written in `daily_notes.md`
- ☐ One git commit with a descriptive message

Daily Commit Examples

```
day08: implement axi-lite independent aw/w channel
      capture
day15: add axi burst write memory with wlast
      checking
day20: debug intentional axi valid stability
      violation
```

Waveform Debug Prompts

Question	Evidence to capture
Where is the transaction accepted?	The exact clock edge where both VALID and READY are high.
What stalled?	Which READY or VALID held the transaction back, and for how long.
Did payload remain stable?	Side-by-side signal view during every stall cycle.
Did reset recover cleanly?	Post-reset outputs, FSM state, and absence of X values.
Did the assertion point to the bug?	Assertion name, failure time, and linked waveform location.

What Not To Do

- Do not read specs passively for hours without coding.
- Do not memorise every CHI field before AXI is solid.
- Do not build a huge unfinished SoC.
- Do not watch tutorials without coding beside them.
- Do not copy GitHub AXI cores as a substitute for writing your own.
- Do not spend days on UVM before SV/SVA basics are sound.
- Do not avoid questions until you feel perfect.
- Do not count reading-only days as preparation.

11. Reading Strategy

Hennessy & Patterson

Use for architecture context only. Do not try to read the whole book in 26 days.

Priority topics

- Memory hierarchy: caches, levels, miss rates
- Cache coherency
- Multiprocessors and interconnects
- Latency, bandwidth, and throughput

Daily pattern

1. Read 5–10 pages tied to today's concept.
2. Extract five concrete ideas.
3. Connect one idea to AMBA/interconnect behaviour.
4. Stop.

Pong P. Chu

Use for RTL craftsmanship. Apply one clean pattern directly to the current block.

Priority topics

- FSM style and reset discipline
- Registers, counters, and memory models
- Parameterised modules
- Modular RTL organisation
- Testbench structure

Application examples

- FSM style → APB slave state machine
- Counter → AXI burst beat counter
- Memory model → AXI4 memory slave
- Modular style → driver/monitor/checker split

Architecture to AMBA Connections

Architecture idea	AMBA connection
Cache coherency	Why ACE and CHI exist.
Memory bandwidth	Why AXI bursts improve transfer efficiency.
Latency hiding	Why outstanding transactions and independent channels matter.
Interconnect contention	Why arbitration, QoS, and backpressure exist.
Reset and timing	Why waveform evidence matters more than assumptions.

12. What “90% Prepared” Means

AMBA

You can explain APB setup/access timing, AHB pipelining, AXI five-channel independence, VALID/READY rules, AXI-Lite read/write flows, AXI bursts, BR-SP/RRESP basics, IDs and same-ID ordering, why ACE/CHI exist, what a snoop is, and CHI node/message vocabulary.

SystemVerilog

You can write interfaces with modports, packages, synthesisable FSMs, parameterised modules, packed structs, driver tasks, passive monitors, basic scoreboards, concurrent SVA properties, and clean synchronous reset logic.

Waveforms

You can debug stalled handshakes, data stability violations, wrong WLAST/RLAST, missing responses, deadlocked VALID/READY, incorrect APB phase timing, W-before-AW write cases, and reset during active transaction.

Emulation Mindset

You understand synthesisable versus simulation-only code, why transactors exist, why waveforms/logs/assertions dominate debug, and why internal team flows must be learned on the job.

Final 26-Day Deliverables

#	Deliverable
1	VALID/READY source, sink, and skid buffer.
2	APB register block with checker.
3	AXI-Lite register block with write and read paths.
4	AXI-Lite to APB bridge.
5	AXI4 burst memory model with write and read support.
6	AXI4 protocol checker with assertions.
7	AHB-Lite mini slave.
8	Toy cache coherency model using MESI states.
9	Toy CHI-style message model.
10	notes/waveform_debug_log.md with annotated traces.
11	notes/amba_cheatsheet.md in your own words.
12	notes/sv_cheatsheet.md in your own words.
13	notes/questions_for_anurag.md for first-week questions.
14	Clean GitHub repo with descriptive daily history.

Final standard. A clean repo with real deliverables is stronger evidence than saying you studied protocols. Evidence is code, simulation logs, waveforms, assertions, notes, and commits.